

PulseGuard AI

Offline-First Maternal Healthcare Platform

A multi-role AI-powered healthcare ecosystem designed for low-bandwidth and offline environments, supporting Patients (Mothers), Health Workers, and Hospital Administrators through predictive analytics, local AI inference, and seamless offline synchronisation.

Submission: Infinity AI BuildFest 2026
Stack: PERN + Python AI Layer
Architecture: Microservices via Docker Compose
AI Models: Llama 3 (local) + Gemini (cloud) + XGBoost
Database: PostgreSQL + pgvector
Language: Bengali + English support

Contents

1	Project Overview	3
1.1	Core Design Principles	3
1.2	User Roles	3
2	System Architecture	3
2.1	High-Level Architecture Map	4
2.2	Backend Service Topology	4
2.3	Docker Service Configuration	5
3	Frontend: UI/UX Flow	6
3.1	Design System	6
3.2	Global Components	6
3.3	Authentication Flow	6
3.4	Patient (Mother) Flow	6
3.4.1	Dashboard	6
3.4.2	AI Symptom Checker	7
3.4.3	Daily Health Check	7
3.4.4	Nutrition Page	7
3.5	Health Worker Flow	7
3.5.1	Dashboard	7
3.5.2	Patient List	7
3.5.3	Patient Details	7
3.5.4	Vitals Entry	7
3.5.5	AI Analysis Page	7
3.5.6	Sync Center	7
3.6	Admin Dashboard (Web – 1440px)	8
3.6.1	Overview Dashboard	8
3.6.2	Patient Management	8
3.6.3	AI Analytics Page	8
3.6.4	Settings Page	8
3.7	Notification System	8
4	Backend: Architecture & Implementation	9
4.1	Auth Service	9
4.2	Patient Records API	9
4.3	Sync Engine	9
4.3.1	Sync Flow	9
4.3.2	Conflict Resolution Strategy	10
4.4	AI Gateway	10
5	AI & Machine Learning Stack	10
5.1	Local Offline LLM	11
5.2	Risk Prediction Model (XGBoost)	11
5.3	Cloud RAG Pipeline (Online Only)	11
6	Database Design	11
6.1	Relational Schema (PostgreSQL)	11
6.2	Data Integrity Measures	12

7	Offline-First System Design	13
7.1	Client-Side Offline Layer	13
7.2	Phase-by-Phase Sync Protocol	13
8	Figma Component System	13
8.1	Component Hierarchy (Atomic Design)	14
8.2	Responsive Breakpoints	14
8.3	Auto Layout Rules	14
9	BuildFest Demo Flow	14
10	Alignment Summary	15

1. Project Overview

PulseGuard AI is an offline-first, multi-role healthcare platform built specifically for low-bandwidth and connectivity-challenged environments. Its primary focus is maternal and prenatal care, enabling health workers in rural areas to collect patient data, run AI-powered risk assessments, and synchronise records seamlessly once a connection is restored.

1.1. Core Design Principles

- **Offline-first:** Every user action works without an internet connection. Data is staged locally and synchronised in the background when connectivity returns.
- **Role-based access:** Three distinct user tiers – Patient (Mother), Health Worker, and Administrator – each with strictly isolated data access enforced at the API level.
- **AI-assisted decisions:** Local LLM inference handles symptom checking and Q&A offline; cloud models handle complex longitudinal summaries when online.
- **Data integrity:** Shared Zod schemas between the React frontend and Express backend ensure offline-queued data always matches database expectations on sync.
- **Bengali language support:** The AI assistant supports Bengali-language input and output using a Bengali-capable local model (e.g., a fine-tuned Llama 3 variant) alongside English, reflecting the target deployment region.

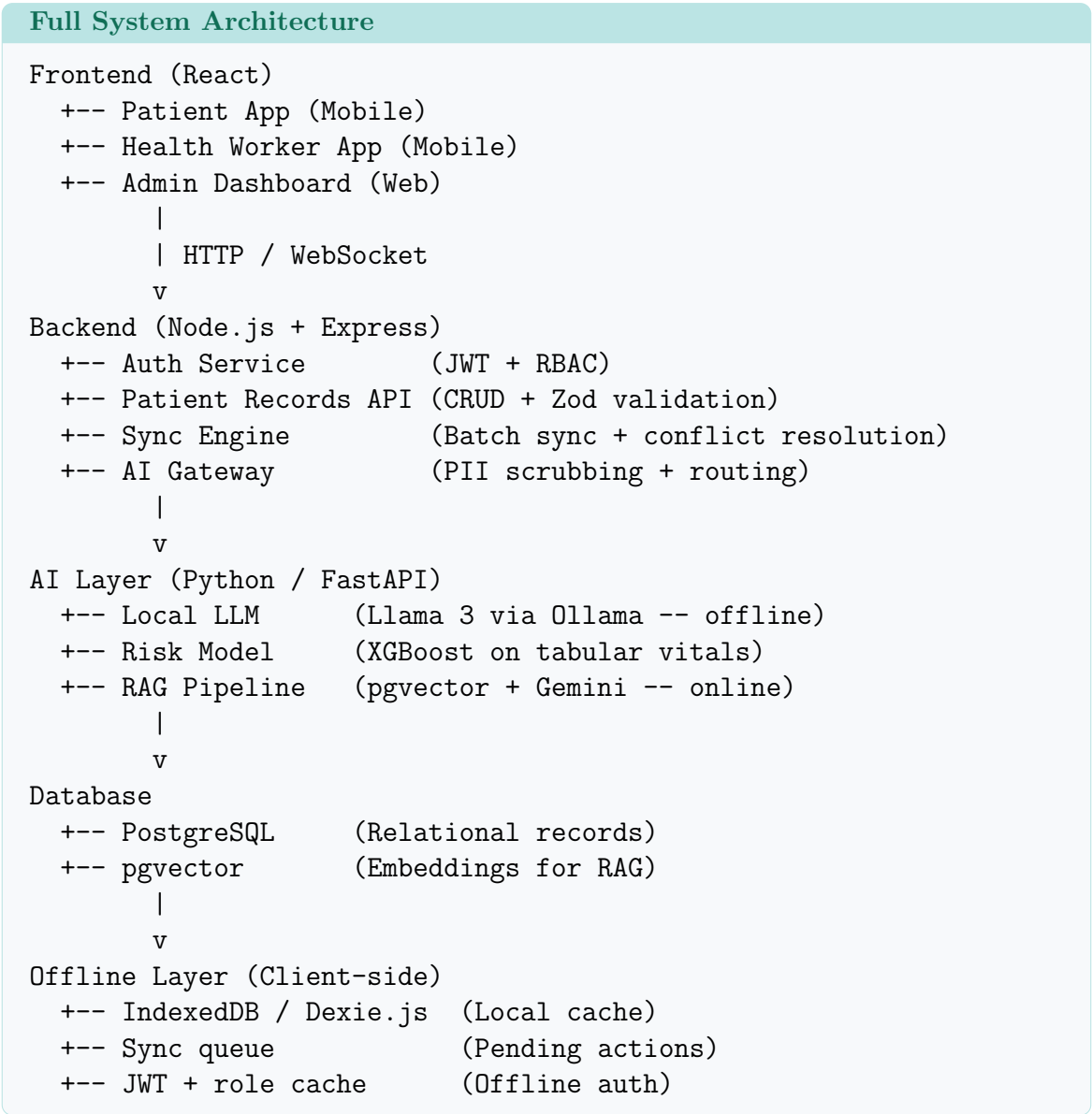
1.2. User Roles

Role	Primary Use	Key Capabilities
Patient (Mother)	Personal health tracking	Health check, AI symptom chat, nutrition
Health Worker	Field patient management	Vitals entry, AI risk analysis, sync center
Administrator	Oversight and analytics	Dashboard KPIs, patient management, analytics

2. System Architecture

PulseGuard is built on a **PERN stack** (PostgreSQL, Express, React, Node.js) with a dedicated Python AI inference layer. All services are orchestrated as isolated microservices using Docker Compose.

2.1. High-Level Architecture Map



2.2. Backend Service Topology

Service	Technology	Responsibilities
Auth Service	Node.js / Express	JWT generation, refresh token rotation, RBAC enforcement. All endpoints validate the JWT and role claim before allowing access. Offline login is supported by caching the JWT and the decoded role payload in IndexedDB; expiry is re-validated on reconnection.
Patient Records API	Node.js / Express	Full CRUD for patient profiles, vitals, visit histories, and pregnancy tracking data. All inputs validated with shared Zod schemas.

Service	Technology	Responsibilities
Sync Engine	Node.js / Express	Receives batched offline payloads at <code>POST /api/sync</code> . Runs Zod validation, checks <code>last_updated</code> timestamps for conflict detection, applies last-write-wins or flags for manual resolution, and exposes conflicts at <code>GET /api/sync/conflicts</code> .
AI Gateway	Node.js / Express	Routes requests to either the local Python AI layer or Gemini. Scrubs all PII from payloads before any external routing.
AI Engine	Python / FastAPI	Hosts the local LLM (Ollama), XGBoost inference, and the RAG ingestion and retrieval pipeline.

2.3. Docker Service Configuration

Listing 1: docker-compose.yml service structure

```

services:
  db:
    image: pgvector/pgvector:pg16
    environment:
      POSTGRES_DB: pulseguard
      POSTGRES_USER: pg_user
      POSTGRES_PASSWORD: pg_secret
    volumes:
      - pg_data:/var/lib/postgresql/data

  api:
    build: ./backend
    ports: ["5000:5000"]
    depends_on: [db]
    environment:
      DATABASE_URL: postgres://pg_user:pg_secret@db:5432/pulseguard
      JWT_SECRET: <secret>

  ai-worker:
    build: ./ai
    ports: ["8000:8000"]
    depends_on: [db]
    volumes:
      - ./models:/models # Local LLM weights

  frontend:
    build: ./frontend
    ports: ["3000:3000"]
    depends_on: [api]

```

3. Frontend: UI/UX Flow

The frontend is a React application with a mobile-first design for Patient and Health Worker roles, and a full-width web dashboard for Administrators.

3.1. Design System

Token	Value	Usage
Primary (Teal)	#1BA6A6	Buttons, active states, headers
Success (Green)	#4CAF50	Online status, low-risk badges
Warning (Amber)	#FFB020	Moderate risk, syncing state
Danger (Red)	#E53935	High risk, offline, emergencies
Background	#F7F9FB	Page background
Spacing grid	8px base	4 / 8 / 16 / 24 / 32 / 48px
Card radius	12px	All card components
Button radius	16px	All button components

3.2. Global Components

The following components are shared across all three role dashboards:

- **Top Navigation Bar:** Logo, notification bell, connectivity status badge (Online / Syncing / Offline), and profile avatar.
- **Offline Status Banner:** Appears below the navbar when offline, showing the pending sync queue count and a retry button.
- **Status Badge:** Colour-coded pill – green (online), yellow (syncing), red (offline), with the same semantic colours applied to patient risk levels.
- **Bottom Navigation (Mobile):** Home — AI Chat — Records — Alerts — Profile

3.3. Authentication Flow

Entry sequence: Splash Screen → Role Selection → Login / Register → Role-Based Dashboard

Offline Login Strategy

When a user selects *Login Offline Mode*, the app reads the cached JWT and decoded role payload from IndexedDB (written on the last successful online login). The UI renders the appropriate role dashboard from local data. On reconnection, the token is re-validated against the Auth Service; if expired, the user is prompted to re-authenticate before the next sync cycle proceeds.

Login options: **Login Online** (standard JWT flow) — **Login Offline Mode** (cached token). Supported roles: Patient (Mother), Health Worker, Administrator.

3.4. Patient (Mother) Flow

3.4.1. Dashboard

- Greeting header with pregnancy week tracker and risk level badge.
- Quick action cards: Daily Health Check, AI Symptom Checker, Nutrition Plan.

- Health Summary widget: blood pressure, weight, baby movement status, last checkup date.

3.4.2. AI Symptom Checker

Chat interface powered by the local LLM offline and Gemini online. Features:

- Chat bubbles (user right-aligned, AI left-aligned).
- Text input field with quick suggestion chips (Headache, Fever, Dizziness, Nausea).
- AI output format: possible causes, risk level, recommended action.
- Full Bengali and English language support.

3.4.3. Daily Health Check

Form inputs: Blood Pressure, Weight, Temperature, Symptoms, Baby movement. Actions: **Save** (online) — **Save Offline** (queues to IndexedDB sync table).

3.4.4. Nutrition Page

Sections: Recommended Foods, Foods to Avoid, Local Affordable Foods. Filters: High Iron, Protein Rich, Low Cost, Pregnancy Safe. Content sourced from the RAG pipeline (WHO nutrition guidelines + local food databases indexed in pgvector).

3.5. Health Worker Flow

3.5.1. Dashboard

KPI cards: Today's Visits, High Risk Patients, Pending Sync Data, Active Alerts. Quick actions: Patient List, Add Patient, AI Assistant, Sync Center.

3.5.2. Patient List

Search bar with filters (High Risk, Normal, Offline). Each patient card shows name, village, pregnancy week, and a colour-coded risk badge. Actions: View Details, Start Visit.

3.5.3. Patient Details

Tabbed layout: Overview — Vitals — History — AI Insights — Nutrition — Visits.

The AI Insights tab shows the current XGBoost risk score, contributing factors, and the recommended action generated by the AI Gateway.

3.5.4. Vitals Entry

Inputs: Blood Pressure, Weight, Pulse, Temperature, Symptoms. Actions: **Save** — **Save Offline** — **Run AI Analysis** (triggers XGBoost inference via AI Gateway).

3.5.5. AI Analysis Page

Outputs: Risk Prediction (e.g., Preeclampsia Risk %), Malnutrition Alerts, risk explanation. Actions: Generate Report (PDF export via the backend report endpoint), Notify Admin (sends an immediate WebSocket push to the Admin Dashboard).

3.5.6. Sync Center

Displays: pending uploads queue, last sync timestamp, conflict count.

Conflict Resolution UI

A **Resolve Conflicts** button calls `GET /api/sync/conflicts`, which returns flagged records where both the health worker's offline edit and an admin-side edit arrived for the same patient record. The UI presents a side-by-side diff (offline version vs. server version) and lets the worker choose which to keep, sending the decision to `POST /api/sync/conflicts/:id/resolve`.

3.6. Admin Dashboard (Web – 1440px)

3.6.1. Overview Dashboard

KPI widgets: Total Patients, High Risk Cases, Active Health Workers, Offline Devices.

Analytics charts: Maternal risk trends (time-series line chart), Nutrition deficiencies (bar chart), Regional heatmaps (choropleth from patient GPS coordinates stored at visit time).

Real-time alert panel: High-risk notifications pushed via WebSocket from the Sync Engine whenever the XGBoost model crosses a risk threshold.

3.6.2. Patient Management

Table columns: Patient ID, Name, Risk Level, Assigned Worker, Last Visit. Actions: View Profile, Export Data (CSV via `GET /api/patients/:id/export`), Notify Worker (sends a WebSocket message to the assigned worker's active session).

3.6.3. AI Analytics Page

Filters: Region, Date Range, Risk Type. Visuals: Risk trend graphs, Prediction charts. Data sourced from pre-computed aggregations stored in PostgreSQL and refreshed on each successful sync.

3.6.4. Settings Page

Sections: User Management, Roles & Permissions, AI Configuration (model selection, risk thresholds), Offline Settings (sync interval, conflict resolution mode), Language Settings.

3.7. Notification System

Notification Type	Delivery	Trigger
Emergency alert	WebSocket (real-time)	Patient crosses critical risk threshold
Risk detection	WebSocket (real-time)	XGBoost flags a high-risk vitals record
Sync completion	In-app toast	Sync Engine finishes a batch upload
Worker alert	WebSocket (real-time)	Admin presses "Notify Worker" button

Real-Time Alerts for Health Workers

Both the Admin Dashboard **and** connected Health Worker sessions receive live WebSocket alerts when a risk threshold is crossed. The Sync Engine maintains WebSocket connections for all active authenticated sessions. Workers who are offline receive the alert as an in-app notification on their next reconnection and sync cycle.

4. Backend: Architecture & Implementation

4.1. Auth Service

Handles all authentication and authorisation:

- **JWT issuance:** On successful login, issues a signed access token (15-minute expiry) and a refresh token (7-day expiry, stored in an `HttpOnly` cookie).
- **RBAC middleware:** Every protected route runs a middleware that decodes the JWT, checks the `role` claim against a per-endpoint allow-list, and rejects mismatched roles with 403 `Forbidden`.
- **Offline token caching:** On successful online login, the frontend writes the JWT and the decoded role payload to IndexedDB. Offline sessions read from this cache. On reconnection, the token is re-validated; expired tokens trigger a silent refresh if the refresh token is still valid, or a full re-login prompt.

4.2. Patient Records API

RESTful CRUD endpoints for all patient data:

Listing 2: Core patient API routes

GET	/api/patients	-- list (Health Worker: own patients; Admin: all)
POST	/api/patients	-- create patient profile
GET	/api/patients/:id	-- fetch single patient + vitals
PUT	/api/patients/:id	-- update patient profile
POST	/api/patients/:id/vitals	-- add vitals entry
GET	/api/patients/:id/history	-- full visit + vitals history
GET	/api/patients/:id/export	-- CSV export of patient data

All request bodies are validated against shared Zod schemas that are also imported by the React frontend's form components, guaranteeing structural consistency between offline-queued payloads and what the database expects.

4.3. Sync Engine

The Sync Engine handles the offline-to-online data handshake.

4.3.1. Sync Flow

1. The React frontend maintains a global connectivity listener. When the status flips to online, it reads all pending records from the IndexedDB sync queue.
2. The frontend sends a single `POST /api/sync` request with an array of queued action

- objects.
3. The Express controller validates each object with Zod, rejecting any that have been corrupted in local storage.
 4. For each valid record, the engine compares the incoming `last_updated` timestamp with the value in PostgreSQL.
 5. If no conflict: the record is written directly to the database.
 6. If a conflict is detected: the record is written to a `sync_conflicts` table and flagged for manual resolution via `GET /api/sync/conflicts`.
 7. On successful write of new vitals, the Sync Engine emits an internal event to the Python AI layer to trigger XGBoost inference.

Listing 3: Sync Engine core routes

POST	/api/sync	-- batch upload offline
	queue	
GET	/api/sync/conflicts	-- list unresolved
	conflicts	
POST	/api/sync/conflicts/:id/resolve	-- submit resolution
	(keep_local keep_server)	

4.3.2. Conflict Resolution Strategy

Scenario	Resolution
No concurrent edit	Write directly; no conflict recorded.
Admin edited while worker offline	Flag to <code>sync_conflicts</code> ; surface in Sync Center UI for manual resolution.
Identical values	De-duplicate; update <code>last_updated</code> , no conflict.
Non-overlapping fields	Merge fields automatically; mark as auto-resolved.

4.4. AI Gateway

Routes AI requests from the frontend to either the local Python layer or external cloud APIs:

- **Online path:** Request → PII scrub → pgvector RAG retrieval → Gemini for summarisation → response.
- **Offline path:** Request → local Ollama endpoint on the device → Llama 3 inference → response.
- All payloads are scrubbed of patient identifiers (name, ID, location) before being forwarded to any external API.

5. AI & Machine Learning Stack

5.1. Local Offline LLM

- **Model:** A Bengali-capable fine-tune of Llama 3 (e.g., Llama-3-Bengali-8B or equivalent) served via Ollama on the field device.
- **Use cases:** Offline AI symptom checker, maternal Q&A in Bengali and English, quick suggestion chips in the chat interface.
- **Hardware target:** Runs on mid-range Android devices or a shared field laptop using Llama.cpp quantised (Q4_K_M) weights.
- **Language handling:** The model is prompted with a system instruction specifying Bengali as the response language when the user's input is detected as Bengali; English otherwise. No separate translation layer is required.

5.2. Risk Prediction Model (XGBoost)

- **Input features:** Systolic BP, diastolic BP, weight, temperature, pulse, gestational week, recorded symptoms (one-hot encoded), visit frequency delta.
- **Output:** A continuous risk score (0–1) mapped to Low / Moderate / High risk tiers, with a separate binary flag for preeclampsia risk (> 0.7 threshold).
- **Training data:** Synthetically generated tabular records matching the clinical distribution of maternal health data, created to protect patient PII.
- **Inference trigger:** Called automatically by the Sync Engine after each successful vitals write, and on-demand from the Health Worker's "Run AI Analysis" button.
- **Hosting:** Python FastAPI endpoint at POST `/ai/predict` inside the `ai-worker` Docker container.

5.3. Cloud RAG Pipeline (Online Only)

- **Vector store:** pgvector extension within the same PostgreSQL instance, storing embeddings of WHO maternal health guidelines and regional nutrition tables.
- **Retrieval:** On an online AI request, the AI Gateway queries pgvector for the top-k semantically relevant guideline chunks using cosine similarity.
- **Generation:** Retrieved chunks are injected into a Gemini prompt as context for high-quality, grounded summarisation of complex patient histories.
- **Ingestion:** A scheduled Python script parses new guideline PDFs, generates embeddings, and upserts them into pgvector.

6. Database Design

6.1. Relational Schema (PostgreSQL)

Listing 4: Core table definitions

```
-- Users (all roles)
CREATE TABLE users (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name        TEXT NOT NULL,
  phone       TEXT UNIQUE,
  role        TEXT CHECK (role IN ('patient', 'worker', 'admin'))
              NOT NULL,
```

```

    created_at  TIMESTAMPTZ DEFAULT now()
);

-- Patient profiles
CREATE TABLE patients (
    id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id     UUID REFERENCES users(id),
    assigned_worker UUID REFERENCES users(id),
    village     TEXT,
    gestational_week INT,
    risk_level  TEXT DEFAULT 'low',
    last_updated TIMESTAMPTZ DEFAULT now()
);

-- Vitals entries
CREATE TABLE vitals (
    id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    patient_id  UUID REFERENCES patients(id),
    recorded_by UUID REFERENCES users(id),
    bp_systolic INT,
    bp_diastolic INT,
    weight_kg   NUMERIC(5,2),
    temperature_c NUMERIC(4,1),
    pulse       INT,
    symptoms    TEXT[],
    risk_score  NUMERIC(4,3),
    last_updated TIMESTAMPTZ DEFAULT now()
);

-- Sync conflict log
CREATE TABLE sync_conflicts (
    id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    patient_id  UUID REFERENCES patients(id),
    local_payload JSONB,
    server_payload JSONB,
    resolved    BOOLEAN DEFAULT false,
    created_at  TIMESTAMPTZ DEFAULT now()
);

-- pgvector embeddings (RAG)
CREATE TABLE guideline_chunks (
    id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    source      TEXT,
    chunk       TEXT,
    embedding   VECTOR(1536)
);
CREATE INDEX ON guideline_chunks USING ivfflat (embedding
    vector_cosine_ops);

```

6.2. Data Integrity Measures

- All foreign keys are enforced at the database level.
- Zod schemas are the single source of truth for field types and constraints, shared between the frontend forms and backend controllers.
- The `last_updated` column on every mutable table drives conflict detection in the Sync Engine.

- The XGBoost model is trained on synthetic data only; no real patient PII ever leaves the production database.

7. Offline-First System Design

7.1. Client-Side Offline Layer

1. **Local database:** IndexedDB managed via Dexie.js, with tables mirroring the server schema for patients, vitals, and the sync queue.
2. **Sync queue:** Every form submission while offline is written to a `pendingSync` table with a `status: 'queued'` flag instead of hitting the API.
3. **Connectivity listener:** A global React context monitors `navigator.onLine` and the Network Information API. On transition to online, it triggers the background sync service.
4. **Auth cache:** JWT and decoded role stored in IndexedDB on successful login, enabling offline dashboard access without re-authentication.

7.2. Phase-by-Phase Sync Protocol

#	Phase	Detail
1	Offline capture	User inputs data; saved to IndexedDB <code>pendingSync</code> table.
2	Connection event	<code>navigator.onLine</code> fires; sync service reads the queue.
3	Batch upload	<code>POST /api/sync</code> with the full pending array.
4	Zod validation	Backend validates each object; malformed items rejected with detail.
5	Conflict check	<code>last_updated</code> compared; conflicts written to <code>sync_conflicts</code> .
6	AI trigger	Sync Engine emits internal event; XGBoost inference runs on new vitals.
7	WebSocket push	If risk threshold crossed, WebSocket alert sent to Admin + Worker.
8	Queue clear	Successfully synced items removed from local IndexedDB queue.

8. Figma Component System

8.1. Component Hierarchy (Atomic Design)

Level	Components
Atoms	Button (Primary / Secondary / Danger), Input Field, Badge, Icon
Molecules	Patient Card, Alert Card, KPI Card, AI Chat Bubble, Sync Progress
Organisms	Top Navbar, Bottom Navigation, Sidebar (Admin), Dashboard Sections
Templates	Mobile Dashboard Layout, Admin Dashboard Layout
Pages	Patient App, Health Worker App, Admin Web Dashboard

8.2. Responsive Breakpoints

- **Mobile (primary):** 390 × 844 px – Patient and Health Worker apps
- **Tablet:** 768 × 1024 px – adaptive layout
- **Desktop:** 1440 × 900 px – Admin Dashboard only

8.3. Auto Layout Rules

- All screens use Auto Layout (no fixed positioning except icons).
- 8 px spacing grid enforced throughout: 4 / 8 / 16 / 24 / 32 / 48 px.
- Cards are the primary layout unit; no raw text blocks.
- Components use Variants instead of duplicated instances.
- Mobile-first scaling applied to all components before desktop adaptation.

9. BuildFest Demo Flow

The following seven-step narrative is designed as the live presentation storyline:

1. **Patient enters symptoms** – A mother opens PulseGuard on a device with no internet connection. She types dizziness and headache into the AI symptom checker.
2. **AI detects risk (offline)** – The local Llama 3 model responds in Bengali, identifying possible low blood pressure or early preeclampsia indicators and recommending that she inform her health worker.
3. **Health worker records vitals** – The assigned health worker visits the patient and enters her blood pressure (140/90), weight, and pulse. She presses *Save Offline*; the data queues in IndexedDB.
4. **Data saved offline** – The Sync Center shows “Pending Uploads: 1” and the progress bar in a paused state, confirming the data is safely staged locally.
5. **Internet restored** – The connectivity badge transitions from red (Offline) to yellow (Syncing). The background sync service fires automatically.
6. **Sync and AI analysis** – The Sync Engine validates and writes the vitals to PostgreSQL. The XGBoost model evaluates the record and returns a high-risk

score (0.82) for preeclampsia. A WebSocket alert is pushed instantly to the Admin Dashboard and the health worker's session.

7. **Admin sees analytics update** – The Admin Dashboard's real-time alert panel shows the new emergency case. The "High Risk Cases" KPI widget increments. The admin presses *Notify Worker* to confirm the alert.

10. Alignment Summary

The table below confirms every frontend feature maps to a fully specified backend component with the four previously partial items now resolved.

Frontend Feature	Backend Coverage	Status
Role-based dashboards	Auth Service – JWT + RBAC per-endpoint enforcement	OK
Offline-first data entry	IndexedDB + Dexie.js, pendingSync queue, connectivity listener	OK
Offline login mode	JWT + role cached in IndexedDB; re-validated on reconnect	OK
Sync Center UI	POST /api/sync batch endpoint, Zod validation	OK
Conflict resolution UI	GET /api/sync/conflicts + POST resolve endpoint	OK
AI Symptom Checker	Llama 3 Bengali-capable model via Ollama (offline)	OK
Bengali language support	Bengali-capable Llama 3 fine-tune; system prompt language detection	OK
Risk prediction badge	XGBoost on tabular vitals; POST /ai/predict	OK
Vitals entry form	Patient Records API – CRUD with shared Zod schemas	OK
Nutrition page	RAG pipeline – pgvector + WHO docs + Gemini (online)	OK
Admin real-time alerts	WebSocket push from Sync Engine on risk threshold	OK
Health worker real-time alerts	WebSocket broadcast to all active sessions (admin + worker)	OK
Online/Syncing/Offline badge	navigator.onLine listener triggers background sync service	OK
Longitudinal AI summary	AI Gateway routes to Gemini + pgvector RAG when online	OK
Notify Worker (admin action)	WebSocket message to worker's active session via Sync Engine	OK

Scope Note

The following features were excluded from the project scope to maintain a lean, fully-specified implementation: voice input / speech-to-text, PDF / CSV report export, appointment reminders, file upload in AI chat, password reset flow, and unauthenticated emergency splash access. These can be added as post-BuildFest extensions without architectural changes to the core system.